

chapter ■ — WhatsApp Automation

Developer Handoff: Migrating from AiSensy BSP to Meta Cloud API

Context

chapter ■ currently sends automated WhatsApp messages through **AiSensy** (a WhatsApp Business Solution Provider / BSP). AiSensy wraps Meta's Cloud API and provides a campaign-based interface. The goal of this handoff is to allow a developer to replicate the exact same automations by calling **Meta's Cloud API directly**, removing the BSP layer.

What stays the same: all trigger logic, database flag checks, Supabase Edge Functions, and deduplication guards remain untouched. **What changes:** the single HTTP call that currently points at AiSensy's endpoint will point at the Meta Graph API instead, with a different auth header and request body shape.

Automation Overview

Message	Campaign Name	Trigger Location	Dedup Flag
Invitation	Invite-Only Automation	src/AdminPanel.tsx — handleApprove()	applications.aisensy_invite_sent
Advance Paid	advance_paid+balance	supabase/functions/payu-callback & payu-webhook	applications.aisensy_advance_paid_sent
Cart Abandon.	cart_abandonment	supabase/functions/cart-abandonment (pg_cron every 30 min)	bill_opens.cart_abandonment_sent
Payment Failed	payment_failed	supabase/functions/payu-callback (failure branch)	applications.aisensy_payment_failed_sent

1. Automation 1 — Invitation Message

■ What it does

Sent to a user the moment an admin clicks **Approve** on their application in the admin panel. It invites the person to proceed with the booking.

■ Trigger location

File: src/AdminPanel.tsx

Function: handleApprove()

Trigger chain:

- Admin clicks Approve on an application row
- applications.status is updated to 'invited' in the DB
- AiSensy API is called with the user's phone number and template params

- On success (HTTP 2xx): applications.aisensy_invite_sent = true
- If aisensy_invite_sent is already true: the API call is skipped entirely

■ Template parameters

Parameter	Value / Source	Example
{{1}}	Event name	Chill-pill in Himalayas
{{2}}	Event date (formatted)	Monday, March 5th

■ Deduplication logic

```
// Before calling AiSensy: if (application.aisensy_invite_sent) return; // already sent – skip //
After success: await supabase.from('applications') .update({ aisensy_invite_sent: true }) .eq('id', application.id);
```

■ Phone number format

Phone is stored as 10 digits (no country code) in applications.phone. When calling the API, prepend **91**: e.g. 9940111564 → 919940111564.

2. Automation 2 — Advance Payment Confirmation

■ What it does

Sent immediately after a user successfully completes the advance payment via PayU. Confirms the amount received and tells the user when the balance is due.

■ Trigger locations (two)

Two Edge Functions both call the same fireAdvancePaidWhatsApp() function. The dedup flag ensures only one message is sent even if both fire.

- supabase/functions/payu-callback/index.ts — triggered when user is redirected back from PayU (browser SURL)
- supabase/functions/payu-webhook/index.ts — triggered by PayU's server-to-server webhook (backup in case user closes browser)

Trigger chain:

- User completes payment on PayU's page
- PayU POSTs to payu-callback (browser redirect) and payu-webhook (server-to-server)
- Both verify the PayU hash, update payu_payments.status = 'success'
- Both update applications.status = 'advance_paid'
- Both call fireAdvancePaidWhatsApp() — dedup flag prevents double-send
- Only fires when payment_type = 'advance' (not for balance payments)

■ Template parameters

Parameter	Value / Source	Example
{{1}}	Amount paid (INR formatted)	₹4,500
{{2}}	Balance due date	June 10th
{{3}}	PayU transaction ID	CHA17391234ABCD

■ How the balance due date is calculated

The events table has a booking_steps JSON column — an array of steps. The function scans for the step whose label/value contains the word *balance* (case-insensitive) and formats its date field.

```
const balStep = (ev?.booking_steps ?? []).find((s: any) => /balance/i.test(`${s.label ?? ''}${s.value ?? ''}`) ); // Format: 'June 10th' function formatDueDate(iso: string): string { if (!iso) return ''; const d = new Date(`${iso}T00:00:00`); const month = d.toLocaleDateString('en-US', { month: 'long' }); const day = d.getDate(); const s = ['th','st','nd','rd'], v = day % 100; return `${month} ${day}${s[(v-20)%10] || s[v] || s[0]}`; }
```

Note: If no booking_steps entry has 'balance' in its name, or no date is set, the field sends as empty. Recommend adding a fallback: 'as per event schedule'.

■ Deduplication logic

```
if (application.aisensy_advance_paid_sent) return; // skip // After success: await supabase.from('applications') .update({ aisensy_advance_paid_sent: true }) .eq('id', application.id);
```

3. Automation 3 — Cart Abandonment Message

■ What it does

Sent to a user who opened the bill/payment page but never tapped Pay. Fires approximately 2 hours after their last bill page open.

■ Trigger — pg_cron + Edge Function

File: supabase/functions/cart-abandonment/index.ts

A **pg_cron job runs every 30 minutes** and calls the cart-abandonment Edge Function via HTTP POST.

Trigger chain:

- User reaches the bill page → frontend upserts a row into bill_opens table (phone, event_slug, opened_at = now, cart_abandonment_sent = false)
- If user opens the bill page again: opened_at is reset to the new timestamp (UNIQUE constraint on phone + event_slug)
- pg_cron fires every 30 min → calls cart-abandonment Edge Function
- Function queries: cart_abandonment_sent = false AND opened_at < NOW() - 2 hours
- For each eligible row: checks payu_payments — if ANY row exists (success/failure/pending) → skip (person already attempted payment)
- Fires AiSensy campaign → on success: cart_abandonment_sent = true

■ pg_cron SQL (already set up in Supabase)

```
SELECT cron.schedule( 'cart-abandonment-every-30min', '*/30 * * * *', $$ SELECT net.http_post( url := 'https://txcmismkdtgtsyhbnefx.supabase.co/functions/v1/cart-abandonment', headers := '{"Content-Type": "application/json"}'::jsonb, body := '{}'::jsonb ) $$ );
```

■ Template parameters

Parameter	Value / Source	Example
{{1}}	User's name	Krutesh
{{2}}	Event title	Sunrise at Kovalam

Name fallback chain: applications.name → bill_opens.name → 'there'

■ Force-test URL (without waiting 2 hours)

```
https://txcmismkdtgtsyhbnef.supabase.co/functions/v1/cart-abandonment?force=true&phone=XXXXXXXXXX&secret=chaptera_cron_2025 Note: force mode still respects cart_abandonment_sent = false.
```

■ Deduplication logic

```
// Skip if already sent: if (row.cart_abandonment_sent) continue; // Skip if payment already attempted: const { data: pay } = await supabase.from('payu_payments') .select('id').eq('phone', row.phone).eq('event_slug', row.event_slug).maybeSingle(); if (pay) { markSent(row.id); continue; } // mark true so it's never checked again // After successful send: await supabase.from('bill_opens').update({ cart_abandonment_sent: true }).eq('id', row.id);
```

4. Automation 4 — Payment Failed Message

■ What it does

Sent when a PayU payment attempt fails (card decline, network drop, etc.). Nudges the user to retry. Even if a user fails 3 times, they only receive this message once.

■ Trigger location

File: supabase/functions/payu-callback/index.ts — inside the else branch when status !== 'success'.

Trigger chain:

- User attempts payment but it fails on PayU's page
- PayU POSTs to payu-callback
- payu-callback updates payu_payments.status = 'failure'
- Calls firePaymentFailedWhatsApp()
- Browser is redirected to: /invite?payment_status=failed&txnid;=...

■ Template parameters

Parameter	Value / Source	Example
{{1}}	User's name	Krutesh
{{2}}	Amount attempted	₹1,024

■ Deduplication logic

```
if (application.aisensy_payment_failed_sent) return; // skip // After success: await supabase.from('applications') .update({ aisensy_payment_failed_sent: true }) .eq('id', application.id);
```

What Changes: AiSensy → Meta Cloud API

All business logic, database flags, trigger conditions, and Edge Function structure stay exactly the same. The only change is the HTTP call inside each fireXxxWhatsApp() function.

■ Current call (AiSensy)

```
POST https://backend.aisensy.com/campaign/t1/api/v2 Authorization: Bearer <AISENSY_API_KEY> Content-Type: application/json { "apiKey": "<AISENSY_API_KEY>", "campaignName": "Invite-Only Automation", "destination": "919940111564", "userName": "chapter A 3063", "templateParams":
```

```
["Chill-pill in Himalayas", "Monday, March 5th"] }
```

■ Replacement call (Meta Cloud API)

```
POST https://graph.facebook.com/v19.0/<PHONE_NUMBER_ID>/messages Authorization: Bearer
<META_PERMANENT_TOKEN> Content-Type: application/json { "messaging_product": "whatsapp", "to":
"919940111564", "type": "template", "template": { "name": "invite_only_automation", // snake_case of
template name "language": { "code": "en" }, "components": [{ "type": "body", "parameters": [ {
"type": "text", "text": "Chill-pill in Himalayas" }, { "type": "text", "text": "Monday, March 5th" }
] }] } }
```

Key differences to handle:

- Auth: AiSensy uses a custom API key in the body; Meta uses a Bearer token in the Authorization header
- Endpoint: AiSensy uses a single campaign endpoint; Meta uses a per-phone-number-ID endpoint
- Template name: AiSensy uses the campaign display name; Meta uses the approved template name in snake_case
- Parameters: AiSensy uses a flat templateParams array; Meta wraps each param as { type: 'text', text: '...' } inside components
- Phone format: same in both — country code + 10-digit number, no +, no spaces (e.g. 919940111564)

The developer will need: (1) a Meta Business account with WhatsApp Cloud API access, (2) an approved WABA phone number ID, (3) a permanent system user token, (4) all templates re-submitted and approved under the Meta WABA directly.

Relevant Database Tables

applications

Column	Type	Description
aisensy_invite_sent	BOOLEAN	Dedup flag — invitation message
aisensy_advance_paid_sent	BOOLEAN	Dedup flag — advance paid message
aisensy_payment_failed_sent	BOOLEAN	Dedup flag — payment failed message
status	TEXT	advance_paid / fully_paid / invited / pending
name	TEXT	Used in payment failed + cart abandonment
phone	TEXT	10-digit, no country code

bill_opens

Column	Type	Description
phone	TEXT	10-digit number
event_slug	TEXT	Canonical events.slug
event_title	TEXT	Event title at time of bill open
opened_at	TIMESTAMPZ	Reset every time bill page is reopened
cart_abandonment_sent	BOOLEAN	One-way flag — never reset to false

Unique constraint: (phone, event_slug) — only one row per person per event.

payu_payments

Column	Type	Description
phone	TEXT	10-digit number
event_slug	TEXT	Canonical events.slug
status	TEXT	pending / success / failure
txnid	TEXT	Unique transaction ID (prefixed CHA...)

Key Files Reference

File / Location	Role
src/AdminPanel.tsx	handleApprove() — triggers invitation message
supabase/functions/payu-callback/index.ts	Advance paid trigger (browser SURL) + payment failed trigger
supabase/functions/payu-webhook/index.ts	Advance paid trigger (server-to-server backup)
supabase/functions/cart-abandonment/index.ts	Cart abandonment — called by pg_cron every 30 min
supabase/functions/create-payu-order/index.ts	Creates payu_payments row — abandonment detection starts here

Prepared for chapter ■ developer handoff. All logic is in the Supabase project: txcmismkdttgshbnexf.